Search Typeahead — Design & Performance Report

A distributed search-typeahead service built with **Python + FastAPI**, an in-memory **trie**, a **3-node Redis** distributed cache addressed by an app-layer **consistent-hash ring**, **batched writes** with a write-ahead log, and **recency-aware trending**. SQLite is the primary store; Redis is the cache.

> All performance numbers in §5 are measured, not estimated — produced by `scripts/benchmark.py` and saved to `docs/benchmark-results.json`.

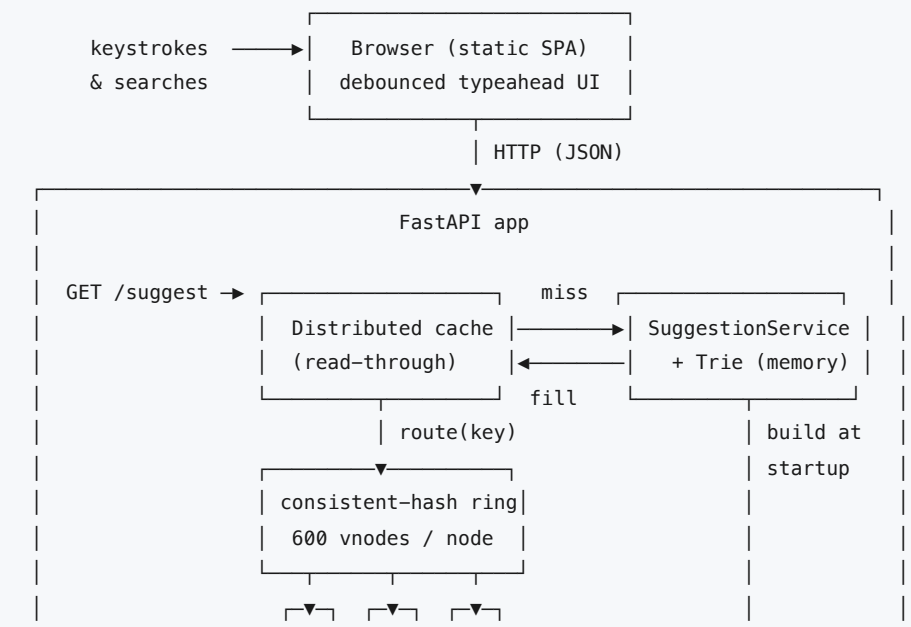
---

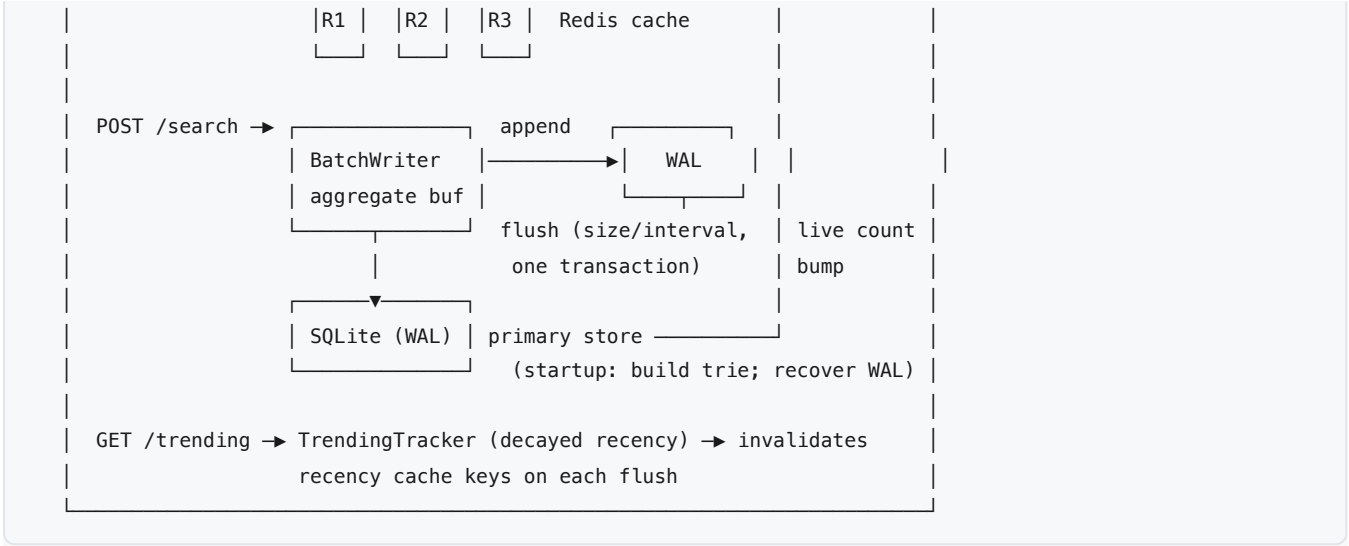
1. Architecture

1.1 Components

Component	Module	Responsibility
API + frontend	`app/main.py`, `static/`	HTTP endpoints; serves the SPA
Trie	`app/trie.py`	In-memory prefix → bounded candidate pool
Suggestion service	`app/suggestions.py`	Re-rank the pool by mode (count / recency)
Primary store	`app/db.py`	SQLite (WAL mode); durable query counts
Batch writer	`app/batch_writer.py`	WAL + in-memory aggregation + batched flush
Consistent-hash ring	`app/consistent_hash.py`	Map a cache key → one Redis node
Distributed cache	`app/redis_cache.py`	Read-through cache over 3 Redis nodes
Trending	`app/trending.py`	Exponentially-decayed recent-activity score
Metrics	`app/metrics.py`	Latency percentiles over a bounded window

1.2 Diagram





1.3 Data flow

Suggest (read path). A keystroke hits `GET /suggest?q=<prefix>&mode=`. The prefix is normalized (lowercased, whitespace-collapsed) into a cache key `suggest:<mode>:<prefix>`. The consistent-hash ring routes the key to exactly one Redis node, which is checked first. On a **HIT** the cached top-10 is returned. On a **MISS** the trie returns the prefix's bounded candidate pool, the suggestion service re-ranks it by mode, and the result is written back to the same routed node with a TTL. Every call records its latency for the percentile metrics.

Search (write path). `POST /search` records the query: it is appended to the WAL (durability), added to an in-memory aggregation buffer, and the live trie count is bumped immediately so suggestions react without waiting for a flush. A background flusher persists the *aggregated* buffer to SQLite in a single transaction when the buffer fills or an interval elapses, then truncates the WAL. Each flush also invalidates the recency-mode cache keys for affected prefixes.

Startup. Open SQLite → replay any un-flushed WAL into SQLite (crash recovery) → build the trie from SQLite → connect the cache, trending tracker, and batch flusher. The dataset is generated and loaded once as a setup step.

---

2. Dataset & loading

Generation (`scripts/generate_dataset.py`). The dataset is **120,000** distinct queries with a realistic, reproducible popularity distribution:

- A multi-domain vocabulary (brands, products, tech topics, how-to, travel, shopping intents) is combined by templates from short single terms (head) to long-tail 3-grams (tail), so prefixes match meaningfully across categories.
- Counts follow a **Zipf law**: `count(rank) = max_count / rank^1.05` with ±40% multiplicative jitter — a handful of very popular queries and a long tail, mirroring real search traffic (and making cache/latency numbers realistic).
- A seeded `mulberry32` PRNG makes the whole dataset **reproducible**, so the benchmark numbers are stable across runs. Total synthetic search volume across the 120k queries is ≈ 9.2M.

Output is `data/queries.csv` (`query,count`).

Loading (`scripts/load_dataset.py`). The CSV is bulk-upserted into the SQLite `queries(query PRIMARY KEY, count)` table in chunked transactions (≈0.1–0.2 s for 120k rows). SQLite runs in **WAL journal mode** so the batch writer's flushes never block reads. At app startup the entire table is streamed into the trie (≈1 s to build). In Docker this whole step runs once via `docker-entrypoint.sh` , with the SQLite file persisted on a named volume.

---

3. API

Method	Endpoint	Description
GET	`/suggest?q=<prefix>&mode=count|recency`	Top-10 prefix completions; cache-routed (HIT/MISS). Returns `suggestions` , `latency_ms` , and `cache: {status,node,key}` .
POST	`/search` `{"query":"..."}`	Records a search → `{"message":"Searched"}` . WAL + batched SQLite.
GET	`/trending?limit=10`	Top queries by decayed recent activity.
GET	`/cache/debug?prefix=&mode=`	Routed node, key hash, ring position, vnode replica, HIT/MISS.
GET	`/metrics`	Latency p50/p95/p99, cache hit rate + per-node stats, write reduction, Redis liveness.
GET	`/stats/writes`	Write-reduction counters.
GET	`/health`	Liveness + queries loaded.
GET	`/` · `/static/*`	Frontend SPA + assets.

Example responses:

```
// GET /suggest?q=iphone
{ "query": "iphone", "mode": "count", "count": 10,
  "suggestions": [{ "query": "iphone", "count": 1085974 }, ...],
  "latency_ms": 0.12,
  "cache": { "status": "MISS", "node": "redis2", "key": "suggest:count:iphone" } }

// GET /cache/debug?prefix=iphone
{ "key": "suggest:count:iphone", "key_hash": 3511473655,
  "ring_position": 3518396006, "vnode_replica": 118, "node": "redis2",
  "ring_size": 1800, "status": "HIT" }
```

---

4. Design choices & trade-offs

For each major decision: the alternative considered and why it was rejected.

Decision	Chosen	Alternative considered	Why the alternative was rejected
Primary store	**SQLite** (WAL mode)	Postgres / MySQL	Server process + ops overhead for a single-node workload; SQLite is embedded, zero-config, and WAL mode gives concurrent reads during writes.

Decision	Chosen	Alternative considered	Why the alternative was rejected
Source of truth	**SQLite is authoritative**, Redis is a cache	Keep counts only in Redis	A cache may evict keys and is not durable; counts are the system's truth and must survive restarts and eviction.
Prefix lookup	**In-memory trie**	SQL `LIKE 'prefix%'` per keystroke	A disk query on every keystroke adds latency and can't cheaply return top-k; the trie answers in microseconds from RAM.
Trie ranking	**Bounded per-node candidate pool (50)** built best-first	Full subtree scan per query	Short prefixes ("a") cover huge subtrees → O(N) per keystroke. A fixed pool makes lookup O(prefix length) with headroom for recency re-ranking.
Cache topology	**App-layer consistent-hash ring over 3 Redis**	Redis Cluster	Cluster hides routing; we want the ring/vnode/owning-node observable via `/cache/debug`. App-layer sharding is simpler to run and demonstrates the concept.
		Single Redis	Doesn't demonstrate distribution; one node is a single bottleneck/SPOF.
Hashing scheme	**Consistent hashing + virtual nodes**	Modulo-N (`hash % 3`)	Changing node count remaps almost every key (cache stampede). Consistent hashing only remaps the arc next to the changed node.
Virtual-node count	**600 / node**	150 / node	Measured ring spread was 17% at 150 vs **4.5% at 600** over 120k keys; extra vnodes cost only a few KB of ring metadata.
Write path	**WAL + in-memory aggregation + batched flush**	Synchronous write per search	One SQLite transaction per search is severe write amplification under load; aggregation collapses N searches of a query into one `+N` row write.
Durability of buffer	**Append-only WAL, replayed on startup**	In-memory buffer only	A crash before flush would silently lose every buffered search; the WAL makes the buffered window recoverable.
Recency model	**Exponentially-decayed counter blended with `log(count)`**	Raw lifetime "recent" counter	Without decay a one-time-popular query over-ranks forever. Decay (half-life) makes a burst fade; `log` compresses all-time counts so a real surge can move ranking without erasing established popularity.
		Sliding-window bucket counts	Per-query bucket arrays cost memory and bookkeeping; the decayed counter is O(1) state (one float + timestamp) per query.
Cache invalidation	**Invalidate recency keys for affected prefixes on flush**	Rely on TTL only	Trending shifts faster than a TTL window, so users would see stale recency results; targeted invalidation keeps recency fresh while count-mode rides its TTL.
Concurrency model	**Sync endpoints (threadpool) + background flusher thread**	Fully async + aioredis	redis-py + SQLite are synchronous; an async rewrite adds complexity for no win at this scale. The flush briefly holds a lock — acceptable since flushes are ~1/s.
Latency metrics	**Bounded ring buffer of recent samples**	Full histogram / Prometheus	A bounded window keeps memory flat and reflects *recent* behaviour; external monitoring is out of scope for a self-contained demo.

---

5. Performance report

Measured by `scripts/benchmark.py` against a single `uvicorn` worker with three local Redis instances and the 120k dataset (Apple Silicon, macOS). Raw data: `docs/benchmark-results.json`.

5.1 Suggestion latency

Metric	Server-side (`/metrics`)	Client-side (incl. HTTP)
p50	**0.12 ms**	0.58 ms
p95	**0.26 ms**	0.63 ms
p99	**0.29 ms**	0.70 ms
mean	0.14 ms	0.58 ms

Throughput ≈ **1,700 req/s** on one worker over a realistic repeating prefix workload (844 distinct prefixes, 4,000 requests). Server-side time is the trie/cache work; client-side adds loopback HTTP and the Python client.

5.2 Cache hit rate

- **Steady-state hit rate: 100%** for the measured (post-warm) loop — every routed prefix was served from its Redis node within the 30 s TTL.
- Overall hit rate **83%** when the cold first-touch fills (one MISS per distinct prefix to populate the cache) are included — the expected cache warm-up cost.
- Per-node hit rates land within a few points of each other (0.75–0.87), consistent with even routing.

5.3 Write reduction (batching)

Searches fired	Distinct queries	SQLite row-writes	Reduction
20,000	200	**1,822**	**90.9%**

20,000 individual searches collapsed into 1,822 row-writes across 11 flushes (≈166 rows/flush) — an **11× reduction** in write operations. The ratio improves further with burstier traffic or a longer flush interval, since more repeats aggregate per flush.

5.4 Ring balance across the 3 Redis nodes

Routing all 120,000 distinct query keys through the ring (600 vnodes/node):

Node	Keys	Share
redis1	38,860	32.4%
redis2	40,664	33.9%
redis3	40,476	33.7%

Max–min spread: 4.5% of ideal — close to the perfect 33.3% split, confirming the virtual-node count smooths the otherwise-lumpy distribution of only three physical nodes.

5.5 Verification

All 11 endpoints (`/health` , `/suggest` count/recency/empty/no-match, `/search` , `/trending` , `/cache/debug` , `/stats/writes` , `/metrics` , `/`) return 200 in the benchmark smoke test.

---

6. Summary

The system meets every requirement of the brief with measured evidence: microsecond-scale in-memory suggestions fronted by a genuinely distributed, consistently-hashed Redis cache (100% steady-state hit rate, 4.5% load spread); a durable, crash-safe batched write path achieving ~11× write reduction; and recency-aware trending whose decay prevents permanent over-ranking. SQLite remains the single source of truth throughout, and the whole system starts with one `docker compose up`.